Wrocław University
of Science and Technology

Optimization Methods
Project Group 1

Task 0: Implementation and Study of Evolutionary Algorithms Compared to Non-evolutionary Methods

The Permutation Flowshop Scheduling Problem
— Project's Report —

Student

Maxime Luce
Student ID: 294620

Teacher

Marcin Komarnicki

Academic year 2025/2026 – March 22, 2026

Contents

1 The Permutation Flowshop Scheduling Problem **2**

1.1 General overview 2

1.2 Specifics 2

1.3 Objectives 2

1.4 Constraints 2

1.5 Chosen problem representation 3

1.6 Solved example 3

2 Theoretical review **4**

2.1 Genetic Algorithms 4

2.2 ds 4

3 Implementation **4**

3.1 Main structure 4

3.2 Random Search 4

3.3 Greedy 4

3.4 EA 4

3.5 Simulated Annealing 4

4 Results and analysis **4**

4.1 notepadd graphics 5

4.2 Evolution 5

Conclusion **5**

Appendixes **6**

List of Tables 6

List of Figures 6

Introduction

This documents aims to present the main results on the *Permutation Flowshop Scheduling Problem* (PFSP).

Context

Presentation of the plan

1 The Permutation Flowshop Scheduling Problem

1.1 General overview

The PFSP is defined by a set of M machines, J jobs and a matrix $T \in \mathcal{M}_{M,J}(\mathbb{R})$, where $t_{m,j}$ represents the processing times of job j on machine m with $j \in [\![1; J]\!]$ and $m \in [\![1; M]\!]$.

The constraints are that each job must be processed on each machine. Moreover, the order of machines on which jobs are processed is predefined, which mean that we cannot start processing a job on a machine until the job is done on the previous machine, and the order is the same for all jobs. Finally, each machine can process only one job at a time.

We aim to find a job sequence $x = \langle x_i, \dots, x_n \rangle$ to be processed on each machine which minimizes the total flow time. We define the total flow time as the sum of the completion times of each job on the last machine. Hence, the objective function f that compute the total flow is defined as:

$$f(x) = \sum_{j=1}^{J} c(x_j, M), \tag{1}$$

where $c(x_j, m)$ is the completion time of the x_j-th job on the $m-$th machine. This quantity can be calculated with

$$c(x_j, m) = \begin{cases} p(x_j, m) & \text{, if } j = 1 \text{ and } m = 1 \\ c(x_j, m-1) + p(x_j, m) & \text{, if } j = 1 \text{ and } m > 1 \\ c(x_{j-1}, m) + p(x_j, m) & \text{, if } j > 1 \text{ and } m = 1 \\ \max\{c(x_{j-1}, m), c(x_j, m-1)\} + p(x_j, m) & \text{, if } j > 1 \text{ and } m > 1 \end{cases}, \tag{2}$$

where $p(x_j, m)$ is the processing time on job x_j on machine m.

1.2 Specifics

the goal sould target a specific area of improvement or answer a specific need

1.3 Objectives

1.4 Constraints

1.5 Chosen problem representation

1.6 Solved example

Let's consider the following problem with 4 jobs and 5 machines and the given computation times matrix (3).

$$P = \begin{pmatrix} 30 & 29 & 19 & 50 \\ 39 & 69 & 6 & 73 \\ 3 & 86 & 58 & 60 \\ 72 & 74 & 51 & 15 \\ 24 & 47 & 22 & 90 \end{pmatrix} \tag{3}$$

For this problem, we know the optimal solution, which is the sequence $x^* = \langle 1, 3, 4, 2 \rangle$.

By using equation (1), we can compute the total flow time:

$$f(x^*) = c(1,5) + c(3,5) + c(4,5) + c(2,5)$$

Now, we use (2) to evaluate each cost. We present the calculations in the reverse order necessary to understand the dependencies between the relationships

- **Calculation of** $c(1,5)$
  - We can in fact prove by recurrence that, for all $n \in \mathbb{N}$, we have $c(1,n) = \sum_{j=1}^{n} p(1,j)$.
  - For $n = 5$, we have indeed:

$$\begin{aligned} c(1,5) &= c(1,4) + p(1,5) \\ &= c(1,3) + p(1,4) + p(1,5) \\ &= \cdots \\ &= p(1,1) + p(1,2) + p(1,3) + p(1,4) + p(1,5) \\ &= 30 + 39 + 3 + 72 + 24 \\ \boxed{c(1,5) = 168} \end{aligned}$$

- **Calculation of** $c(3,5)$
  - $c(1,2) = p(1,1) + p(1,2) = 40 + 39 = 69$
  - $c(3,1) = c(1,1) + p(3,1) = p(1,1) + p(3,1) = 30 + 19 = 49$
  - $c(3,2) = \max\{c(1,2), c(3,1)\} + p(3,2) = c(1,2) + p(3,2) = 69 + 6 = 75$
  - $c(3,3) = \max\{c(1,3), c(3,2)\} + p(3,3) = c(3,2) + p(3,3) = 75 + 58 = 133$
  - $c(3,4) = \max\{c(1,4), c(3,3)\} + p(3,4) = c(1,4) + p(3,4) = 144 + 51 = 195$

- $c(3,5) = \max\{c(1,5), c(3,4)\} + p(3,5) = c(3,4) + p(3,5) = 195 + 22 = 217$
- $\boxed{c(3,5) = 217}$

- **Calculation of** $c(4,5)$
  - $c(4,1) = c(3,1) + p(4,1) = 49 + 50 = 99$
  - $c(4,2) = \max\{c(3,2), c(4,1)\} + p(4,2) = c(4,1) + p(4,2) = 99 + 73 = 172$
  - $c(4,3) = \max\{c(3,3), c(4,2)\} + p(4,3) = c(4,2) + p(4,3) = 172 + 60 = 232$
  - $c(4,4) = \max\{c(3,4), c(4,3)\} + p(4,4) = c(4,3) + p(4,4) = 232 + 15 = 247$
  - $\boxed{c(4,5) = 247}$

- **Calculation of** $c(2,5)$
  - $c(2,1) = c(4,1) + p(2,1) = 99 + 29 = 128$
  - $c(2,2) = \max\{c(4,2), c(2,1)\} + p(2,2) = c(4,2) + p(2,2) = 172 + 69 = 241$
  - $c(2,3) = \max\{c(4,3), c(2,2)\} + p(2,3) = c(2,2) + p(2,3) = 241 + 86 = 327$
  - $c(2,4) = \max\{c(4,4), c(2,3)\} + p(2,4) = c(2,3) + p(2,4) = 327 + 74 = 401$
  - $c(2,5) = \max\{c(4,5), c(2,4)\} + p(2,5) = c(2,4) + p(2,5) = 401 + 47 = 448$
  - $\boxed{c(2,5) = 448}$

- Finally, we get $\boxed{f(x^*) = 168 + 217 + 337 + 448 = 1170}$

2 Theoretical review

2.1 Genetic Algorithms

2.2 ds

3 Implementation

3.1 Main structure

3.2 Random Search

3.3 Greedy

3.4 EA

3.5 Simulated Annealing

4 Results and analysis

4.1 notepadd graphics

4.2 Evolution

Conclusion

Appendixes

List of Tables

List of Figures